

拓扑 AC CSP-S & NOIP 联测第三轮第五场讲评

袁胜利

2026 年 8 月 17 日

吐槽环节/自由分享

字符串指纹

字符 a 到 z 的值为 0 到 25。对子串 $s_i \cdots s_j$, 定义

$$H(i, j) = \sum_{t=i}^j \text{val}(s_t) 26^{t-i}.$$

每次询问给出 l, r, k , 求满足 $l \leq i \leq j \leq r$ 且 $H(i, j) = k$ 的位置对数量。
 $n, m \leq 10^5$, $0 \leq k \leq 10^{14}$ 。

观察

先看 $k > 0$ 。把 k 写成 26 进制，并按低位到高位还原成字符，就得到它唯一的“核心串”。例如 26 的核心串是 ab。

由于最高位不能是 0，所有指纹等于 k 的字符串都具有形式

核心串 + 若干个 a.

又因为 $26^{10} > 10^{14}$ ，核心串长度至多为 10。因此长度超过 10 的合法字符串，超过第十位的部分一定全是 a。

$k = 0$ 时合法字符串全部由 a 组成，单独统计。

字符串指纹

子任务一 (20 分, $n, m \leq 1000$): 对每次询问枚举起点 $i \in [l, r]$, 沿 “下一处非 a” 指针计算至多十个可能的非零指纹。

若某次指纹等于 k , 设它在结尾 p 首次出现、到 z 之前都不再变化, 则本次贡献为

$$\max(0, \min(z, r) - p + 1).$$

$k = 0$ 时只需统计从 i 开始的连续 a。时间复杂度为 $O(10nm)$ 。

观察

下面预处理所有起点，而不是为每次询问重新枚举。

固定起点 i ，暴力加入长度 1 至 10 的全部子串。若长度为 10 的子串后面还有一段连续的 a ，这些更长的子串指纹不变，把这整段的数量一起加入。

对每个指纹 h ，按起点保存贡献并做前缀和。记

$$F_h(x) = \#\{(i, j) : i \leq x, H(i, j) = h\},$$

这里暂时不限制结尾 j 。相同起点、相同指纹的贡献先合并，再在该指纹对应的起点数组上做前缀和。

总共只会加入 $O(10n)$ 次贡献。

字符串指纹

子任务二 (20 分, 所有询问均满足 $l = 1, r = n$): 询问不再限制起点或结尾。

预处理时已经统计了每个指纹在整个字符串中的出现次数。询问答案就是 $F_k(n)$, 在哈希表中找到 k 后读取最后一个前缀和即可。

每个起点至多产生十条非零记录, 零指纹另有至多一条记录。时间复杂度为 $O(10n + m)$, 空间复杂度为 $O(10n)$ 。

观察

对一般询问 $[l, r]$ ，先计算

$$F_k(r) - F_k(l-1).$$

它选出的起点恰好在 $[l, r]$ 中，但预处理没有限制结尾，因此还多算了

$$l \leq i \leq r < j \leq n$$

的子串。把这些跨过右边界 r 的贡献减掉，才是询问答案。

观察

修正跨过 r 的子串。若 $k = 0$ ，子串必须全是 a ：分别求出边界左侧可选起点数和右侧可选结尾数，两者相乘即可。

以下设 $k > 0$ 。跨边界子串的起点若在 $[r - 9, r]$ ，直接枚举；若起点早于 $r - 9$ ，则从起点第十一位开始必须全为 a ，所以起点只能落在“ r 左侧最后一个非 a ”之前的九个位置内。

因此候选起点至多 20 个。对每个候选起点计算前十位指纹，并用下一处非 a 的位置求出能越过 r 的结尾数量；指纹等于 k 时从答案中扣除。

每个候选起点只检查十位，单次询问的修正量为常数规模。

字符串指纹

子任务三 (60 分, 无额外限制):

- ① 枚举每个起点的前十位, 并把可追加的 a 一并计入各指纹的起点前缀和;
- ② 用 $F_k(r) - F_k(l-1)$ 取得起点在询问区间内的全部贡献;
- ③ 枚举边界附近的常数个候选, 减去结尾超过 r 的贡献。

预处理为 $O(10n)$ 次加入。查询指纹对应的起点前缀和需要二分, 边界修正至多检查约 10 个位置, 因此总时间复杂度为 $O(10n + m(\log n + 10))$ 。

周期航线

道路只决定航线相邻停靠位置之间的行驶时间，旅客不能沿道路单独移动。每条航线按固定站点顺序单向运行，首站发车时刻为

$$x, x + y, x + 2y, \dots$$

从站点 1 在时刻 t 出发，最多换乘 k 次，即最多乘坐 $k + 1$ 条航线，求到达站点 n 的最早时刻。

记所有航线长度之和为 L 。有 $n \leq 10^4$, $m, L \leq 5 \times 10^4$, $s \leq 2.5 \times 10^4$, $k \leq 100$ 。

观察

固定一条航线，依次经过 $v_1, v_2, \dots, v_l$ 。令 d_i 表示船从首站 v_1 行驶到第 i 个停靠位置所需的时间，所以 $d_1 = 0$ 。

若乘客在时刻 T 到达第 i 个停靠位置，记 $B_i(T)$ 为能赶上的最早一班船从首站出发的时刻，则

$$B_i(T) = x + \max \left(0, \left\lceil \frac{T - x - d_i}{y} \right\rceil \right) y.$$

在位置 i 上船、在后面的 j 下船，到达时刻就是 $B_i(T) + d_j$ 。

这里 i, j 是航线中的停靠位置， v_i, v_j 才是站点编号；同一站点可以在一条航线中出现多次。

周期航线

子任务一 (20 分, $n \leq 20$, $m \leq 190$, $s \leq 30$, $k \leq 3$, $L \leq 100$): 令 $f_h(u)$ 表示恰好乘坐 h 条航线后到达站点 u 的最早时刻, 初始 $f_0(1) = t$, 其余为无穷大。从 f_{h-1} 转移到 f_h 时, 枚举每条航线以及所有 $i < j$ 。若 $f_{h-1}(v_i)$ 有限, 就用

$$B_i(f_{h-1}(v_i)) + d_j$$

更新 $f_h(v_j)$ 。

时间复杂度为 $O((k+1)(n + \sum \beta) + m + L)$ 。

一条航线不必枚举全部 $i < j$ 。从左到右扫描停靠位置，处理第 j 个位置前维护

$$best = \min_{i < j} B_i(f_{h-1}(v_i)).$$

扫描到 j 时按下面的顺序操作：

- ① 若 $best$ 有限，用 $best + d_j$ 更新 $f_h(v_j)$ ；
- ② 若 $f_{h-1}(v_j)$ 有限，计算 $B_j(f_{h-1}(v_j))$ 并更新 $best$ 。

先下船、后加入当前位置，保证上车位置严格小于下车位置。这样每条航线只需 $O(l)$ 时间。

周期航线

子任务二 (20 分, 所有航线均满足 $l = 2$): 每条航线只有一个合法位置对, 把它看成一条带周期发车时刻的有向边。

计算第 h 层时, 只能读取 f_{h-1} , 所有更新写入新的 f_h 数组; 否则会在同一层连续乘坐多条航线。

每层为 $O(n + s)$, 总时间复杂度为 $O(m + L + (k + 1)(n + s))$ 。

周期航线

子任务三 (20 分, $k = 0$): 全程最多乘坐一条航线, 只需从

$$f_0(1) = t$$

计算一层转移。对每条航线做一次从左到右扫描; 站点 1 的每次停靠都可成为上车候选。

预处理道路时间与航线前缀后, 时间复杂度为 $O(m + L + n)$, 空间复杂度为 $O(m + L + n)$ 。

周期航线

子任务四 (40 分, 无额外限制): 依次计算 $f_1, f_2, \dots, f_{k+1}$ 。每一层使用独立的新数组, 并对所有航线各线性扫描一次。答案为

$$\min_{1 \leq h \leq k+1} f_h(n).$$

分层保证每次只增加一段乘坐; 线路扫描中的 *best* 汇总所有更早的上车位置, 所以最后一段乘坐的所有选择都被覆盖。

总时间复杂度为

$$O(m + L + (k + 1)(n + L)),$$

空间复杂度为 $O(m + n + L)$ 。所有时刻使用 64 位整数。

重复警戒

统计长度为 n 、每项属于 $1, \dots, k$ 的序列，要求每种编号都出现，并且不存在长度为 k 、元素两两不同的连续段。

$1 \leq k \leq n \leq 5000$ ，答案对 998244353 取模。

观察

先统计每种编号都出现的序列，再减去出现过非法窗口的序列。
任意长度为 k 的互异窗口，恰好包含 $1, \dots, k$ 各一次，所以含非法窗口的序列一定已经使用了全部编号。两个集合存在包含关系，直接相减不会误删。
窗口是否合法只与当前最长互异后缀有关。追加一个编号后，这段后缀要么增长一位，要么从该编号上一次出现位置之后重新开始。

重复警戒

子任务一 (10 分, $n \leq 10$): 编号的具体名称不影响窗口是否合法, 只需枚举各位置的相等关系。

用受限增长串表示集合划分: 首项为 0, 之后每项可以使用已有标号, 也可以使用下一个新标号。最终恰好出现 k 个标号时, 有 $k!$ 种方式映射到真实编号。

若末尾已经出现 k 个互异标号, 或剩余位置不足以补齐 k 类, 就立即剪枝。时间复杂度为 $O(B_n n)$, 其中 B_n 是第 n 个 Bell 数 (将一个包含 n 个元素的集合划分为若干个非空无序子集的方法总数)。

重复警戒

令 $u_{i,j}$ 表示长度为 i 、恰好用过 j 种编号的序列数，初始 $u_{0,0} = 1$ 。
最后一个编号若已经出现，有 j 种选择；若是新编号，要从尚未使用的 $k - j + 1$ 种中选择。因此

$$u_{i,j} = j u_{i-1,j} + (k - j + 1) u_{i-1,j-1}.$$

于是 $u_{n,k}$ 恰好是使用全部编号的序列数。

重复警戒

令 $b_{i,j}$ ($0 \leq j < k$) 表示长度为 i 、尚未出现非法窗口，且最长互异后缀长度为 j 的序列数； $b_{i,k}$ 是已经出现非法窗口的吸收态。初始 $b_{0,0} = 1$ ，其余状态为 0。

对 $1 \leq j < k$:

$$b_{i,j} = (k - j + 1)b_{i-1,j-1} + \sum_{p=j}^{k-1} b_{i-1,p}.$$

第一项给长度 $j-1$ 的后缀追加一个新编号；第二项从长度 $p \geq j$ 的后缀中，追加倒数第 j 个编号。先直接枚举求和中的所有 p 。

重复警戒

子任务二 (30 分, $k \leq 80$): 直接按照定义计算

$$\sum_{p=j}^{k-1} b_{i-1,p}.$$

每个 $b_{i,j}$ 枚举全部 $p \geq j$, 一层有 $O(k)$ 个状态, 每个状态转移 $O(k)$, 时间复杂度为 $O(nk^2)$, 空间复杂度为 $O(k)$ 。

当 $k \leq 80$ 时可以通过。

重复警戒

子任务三（60 分，无附加限制）：对上一层维护后缀和

$$s_j = \sum_{p=j}^{k-1} b_{i-1,p},$$

即可在 $O(1)$ 时间得到每个求和项。已经非法的序列可以追加任意编号；长度为 $k-1$ 的互异后缀只有一个尚未出现的编号，因此

$$b_{i,k} = k b_{i-1,k} + b_{i-1,k-1}.$$

最终答案为

$$u_{n,k} - b_{n,k} \pmod{998244353}.$$

两组 DP 都只读取上一层，使用滚动数组和后缀和后，时间复杂度为 $O(nk)$ 。

顺序取件

货箱按一个排列放在货架上，按编号从小到大永久取走。取目标箱前，必须临时移出它前面的货箱；临时货箱放回前端时可以任意重排。之后依次永久删除若干给定原始位置。输出初始状态以及每次删除后的最少移动次数。

$n \leq 10^5$, $0 \leq q < n$ 。

观察

只看当前仍存在的货箱，记编号 v 的原始位置为 p_v 。

若存在更小编号 $u < v$ 位于 v 右侧，处理 u 时可重排的前缀已经越过 p_v ，可以顺便把 v 放到以后不再受阻的位置。

因此只有满足

$$p_v > \max_{u < v} p_u$$

的货箱会产生额外移动。称它们为纪录点；在当前货架上从右向左扫描时，它们正是严格递减出现的后缀最小值。

顺序取件

设当前还有 m 个货箱。对纪录点 v ，记 $P(v)$ 为它的当前位置， $V(v)$ 为它在剩余编号中的位置。

v 前面共有 $P(v) - 1$ 个货箱，其中 $V(v) - 1$ 个编号比它小；剩下的

$$P(v) - V(v)$$

个较大编号必须被临时移出。所有货箱还各被永久取走一次，所以静态答案为

$$m + \sum_{v \text{ 为纪录点}} (P(v) - V(v)).$$

顺序取件

子任务一 (20 分, $n, q \leq 100$): 每次删除后重新构造剩余序列。
从右向左找出后缀最小值; 同时用剩余位置数得到 $P(v)$, 按编号排序或用频次数组得到 $V(v)$, 代入静态公式。
直接实现总复杂度可写为 $O(n(q+1)\log n)$ 。

顺序取件

子任务二 (20 分, $n, q \leq 1000$): 仍在每次删除后重新计算静态答案。
因为原序列是排列, 可以用布尔数组标记删除, 并在线性扫描中重建剩余编号与后缀最小值, 每个状态为 $O(n)$ 。
总时间复杂度为 $O(n(q+1))$ 。

顺序取件

子任务三 (20 分, $q = 0$): 只有初始状态。

从右向左扫描排列, 维护当前最小编号, 即可找出全部纪录点。此时没有删除, $P(v) = p_v$ 、 $V(v) = v$, 所以直接累加

$$n + \sum_{v \text{ 为纪录点}} (p_v - v).$$

时间复杂度为 $O(n)$ 。

观察

令时间 t 表示前 t 次删除已经发生, $D(v)$ 表示货箱 v 在第几次事件中被删除;
若始终不删, 令 $D(v) = q + 1$ 。

对货箱 v 来说, 所有满足 $u < v$ 且 $p_u > p_v$ 的货箱, 都是让 v 不能成为纪录点的“阻挡者”。只有等这些货箱全部删除后, v 才成为纪录点。

最后一个阻挡者的删除时间为

$$S(v) = \max_{u < v, p_u > p_v} D(u),$$

没有阻挡者时取 $S(v) = 0$ 。所以 v 从时刻 $S(v)$ 开始成为纪录点, 到自身在时刻 $D(v)$ 被删除为止, 即活动区间为

$$[S(v), D(v)).$$

按编号从小到大扫描, 用一棵维护最大值的树状数组查询右侧位置的最大删除时间, 即可求出所有 $S(v)$ 。

顺序取件

时间 t 时, 设:

- $A_t(p)$: 原始位置不超过 p 、并且已经删除的货箱数;
- $B_t(v)$: 编号不超过 v 、并且已经删除的货箱数。

于是货箱 v 当前的位置和编号分别为

$$P_t(v) = p_v - A_t(p_v), \quad V_t(v) = v - B_t(v).$$

所以活跃纪录点 v 的额外贡献是:

$$P_t(v) - V_t(v).$$

用两棵状数组分别维护 A_t 和 B_t ; 在 $S(v)$ 加入 v , 在 $D(v)$ 移除 v 。

顺序取件

删除一个不是纪录点的货箱，其原始位置为 p 、编号为 v 。对任意活跃纪录点 r ：

- 若 $p < p_r$ ，删除发生在 r 左侧， $P_t(r)$ 减少 1；
- 若 $v < r$ ，删除的编号比 r 小， $V_t(r)$ 减少 1。

两项同时变化时， $P_t(r) - V_t(r)$ 不变。真正少贡献 1 的只有

$$r < v \quad \text{且} \quad p_r > p$$

的纪录点。

活跃纪录点的编号顺序与位置顺序相同。设编号小于 v 的最后一个活跃纪录点位于 p' ，受影响的点恰好位于位置区间 $(p, p']$ 。第三棵树状数组按位置维护活跃纪录点数，查询这段数量并从总贡献中减去即可。

顺序取件

子任务四 (40 分, 无附加限制): 先求每个纪录点的活动区间, 再按删除时间处理:

- ① 必要时扣除删除非纪录点造成的区间贡献;
- ② 在位置、编号树状数组中标记删除;
- ③ 处理当前时刻的纪录点离开与加入事件。

每个点至多加入、离开一次, 每次操作均为 $O(\log n)$ 。维护的额外贡献 E_t 加上剩余货箱数 $n - t$ 即为当前答案。

活跃纪录点用有序集合按编号维护, 以找到前驱位置 p' ; 第三棵树状数组负责统计位置区间内的活跃纪录点数。

总时间复杂度为 $O(n \log n)$ 。